Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura: Programación Orientada a Objetos

Proyecto: Sistema Bancario en Java

Reporte Técnico Final: Implementación de Sistema Bancario (Core Banking)

Equipo de Desarrollo:

- Francisco José Coutiño Morales (Núm. Cta: 425121623)
- Ernesto Flamenco Villaseñor (Núm. Cta: 319170599)
- Jaime Erick Torres Nava (Núm. Cta: 322289976)

26 de marzo de 2026

Índice

1. Introducción **2**

2. Marco Teórico: Jerarquía de Colecciones en Java **2**
2.1. Análisis de Complejidad por Interfaz 2
2.1.1. Listas (List) 2
2.1.2. Conjuntos (Set) 2
2.1.3. Tablas Hash (Map) 2

3. Arquitectura del Modelo de Datos **3**
3.1. Justificación Arquitectural: Desacoplamiento de Entidades 3
3.2. Evolución de Tipos de Dato (De la V1 a la Final) 3

4. Implementación de Estructuras en el Código **3**
4.1. Uso de Map (Directorio Global) 3
4.2. Uso de Set (Cartera de Productos Únicos) 4
4.3. Uso de LinkedList (Historial Transaccional LIFO) 4

5. Flujo de Trabajo y Lógica de Negocio Transaccional **4**
5.1. Prevención de "Dinero Fantasma" 4
5.2. Explicación de Bloques de Código Clave 4

6. Análisis de Diagrama UML y Trazabilidad **5**
6.1. Cumplimiento y Desviaciones del Modelo Original 5

7. Conclusiones **5**

1. Introducción

El presente documento detalla la arquitectura, las decisiones de diseño y el flujo de desarrollo del proyecto "Sistema Bancario El Bienestar". El sistema fue desarrollado en el lenguaje Java, aplicando rigorosamente los paradigmas de la Programación Orientada a Objetos (POO) y haciendo uso intensivo del *Java Collections Framework* para el manejo eficiente de la información en memoria.

El objetivo principal de esta fase final fue modelar las entidades básicas del dominio bancario, construir un núcleo transaccional capaz de gestionar clientes, cuentas de débito, pagarés de inversión y tarjetas de crédito, y garantizar la integridad de los fondos simulando operaciones reales del sistema financiero.

2. Marco Teórico: Jerarquía de Colecciones en Java

El Java Collections Framework es una arquitectura para almacenar y manipular grupos de objetos de forma eficiente. Estas estructuras están optimizadas a nivel de memoria y algoritmos de búsqueda. La jerarquía se divide principalmente en dos ramas:

- **Interfaz Collection:** De ella heredan las interfaces List y Set.
- **Interfaz Map:** Maneja pares clave-valor.

2.1. Análisis de Complejidad por Interfaz

2.1.1. Listas (List)

Permiten elementos duplicados y mantienen el orden de inserción. Un `ArrayList` utiliza un arreglo dinámico que se redimensiona automáticamente, permitiendo acceso en $\mathcal{O}(1)$. Por otro lado, `LinkedList` utiliza nodos doblemente enlazados, optimizando las inserciones en los extremos.

2.1.2. Conjuntos (Set)

Prohíben la existencia de elementos duplicados. Un `HashSet` está respaldado por una tabla hash, ofreciendo operaciones matemáticas de conjuntos con un rendimiento promedio de inserción y búsqueda de $\mathcal{O}(1)$.

2.1.3. Tablas Hash (Map)

Almacenan información mediante una relación de "Clave-Valor", donde cada clave debe ser única. Mapean las llaves a "cubetas" (*buckets*) en memoria, permitiendo búsquedas virtualmente instantáneas de complejidad $\mathcal{O}(1)$.

Estructura	¿Permite Duplicados?	¿Mantiene Orden?	Acceso a Datos
List	Sí	Sí (por inserción)	Por índice (0, 1, 2...)
Set	No	No (generalmente)	Por iteración o búsqueda
Map	Solo en los valores	No	Por clave única

Cuadro 1: Diferencias operativas entre List, Set y Map.

3. Arquitectura del Modelo de Datos

3.1. Justificación Arquitectural: Desacoplamiento de Entidades

Una de las decisiones de diseño más importantes frente a implementaciones triviales fue la **separación estricta entre el ID del Cliente y el Número de Cuenta**.

En el dominio bancario real, un cliente posee múltiples productos. Al separar los identificadores, la entidad `Cliente` funciona como un expediente que encapsula colecciones de diversas cuentas (Cardinalidad 1:N), permitiendo que el sistema escale. Además, por seguridad, el Número de Cuenta es público (para recibir transferencias), mientras que el ID de Cliente es información sensible.

3.2. Evolución de Tipos de Dato (De la V1 a la Final)

Durante la planificación (V1), el número de cuenta y el ID se introducían como `String`. En esta versión final se ejecutó la refactorización a tipos primitivos enteros (`int`):

- **Restricción Numérica:** Garantiza de manera nativa que el sistema rechace caracteres alfabéticos, evitando excepciones de formato.
- **Eficiencia:** Optimiza el procesamiento numérico y el cálculo de colisiones (*hashing*) dentro del `HashMap` y los `HashSet`.

Para lograr esto, se implementó la clase `java.util.Random` en el `GestorBanco`, la cual asigna automáticamente IDs de 9 dígitos únicos al registrar clientes.

4. Implementación de Estructuras en el Código

Para optimizar el rendimiento y cumplir con las reglas de negocio, se asignó una estructura específica a cada necesidad operativa:

4.1. Uso de Map (Directorio Global)

En la clase `GestorBanco`, se implementó un `Map<Integer, Cliente>`. La llave es el número de cliente (`int`) y el valor es el objeto `Cliente`. Esto permite encontrar cualquier perfil de manera instantánea $\mathcal{O}(1)$ al teclear su ID en el cajero.

4.2. Uso de Set (Cartera de Productos Únicos)

Se utilizaron tres colecciones `HashSet` dentro de la clase `Cliente` para separar lógicamente su `CuentaBasica`, `CuentaInversion` y `TarjetaCredito`. Esto asegura matemáticamente que un cliente no pueda tener dos contratos con el mismo número de folio clonado.

4.3. Uso de LinkedList (Historial Transaccional LIFO)

A diferencia de la V1 (donde se propuso `ArrayList`), el historial de la clase `Movimiento` se refactorizó a `LinkedList<Movimiento>`. Un estado de cuenta requiere mostrar la transacción más reciente arriba. Esto dicta un comportamiento **LIFO** (Last-In, First-Out). Usando el método `addFirst()`, la inserción ocurre en $\mathcal{O}(1)$, desplazando los nodos anteriores más rápido que redimensionando un arreglo estático.

5. Flujo de Trabajo y Lógica de Negocio Transaccional

El desarrollo siguió un enfoque iterativo. Tras programar el menú interactivo con la clase `Scanner`, se detectaron y corrigieron vulnerabilidades críticas en la simulación financiera:

5.1. Prevención de "Dinero Fantasma"

Durante las pruebas de caja blanca, se observó que el sistema permitía abrir contratos de inversión y pagar deudas de tarjetas de crédito sin afectar el saldo del cliente (creando capital de la nada).

Para solucionarlo simulando un banco real, se modificó el flujo del método en `GestorBanco`. Ahora, el sistema exige el número de la `CuentaBasica` origen, ejecuta el método booleano `retirar(monto)`, y **solo si este retorna true** (confirmando liquidez), procede a instanciar el contrato de inversión o a invocar el método `abonarDeuda()` de la tarjeta.

5.2. Explicación de Bloques de Código Clave

A continuación, se detalla la lógica detrás de instrucciones fundamentales:

- `sn.nextLine();`: Usado tras leer variables con `nextInt()` o `nextDouble()`. Java deja el carácter de salto de línea (\n) en el flujo. Limpiar el *buffer* evita que el menú del cajero se rompa al leer el siguiente `String`.
- `return !cuentas.isEmpty();`: Una validación booleana elegante. El banco exige poseer una cuenta básica para poder invertir. En lugar de un `if` largo, se niega el estado del `HashSet`; si no está vacío, devuelve `true`, autorizando la transacción.
- `inversiones.remove(inversionTerminada);`: Al liquidar una inversión, esta instrucción busca la referencia exacta del objeto en memoria dentro del `Set` y lo destruye, cerrando el contrato sin dejar *memory leaks*.

6. Análisis de Diagrama UML y Trazabilidad

A continuación, se presenta el modelo conceptual diseñado para el proyecto:

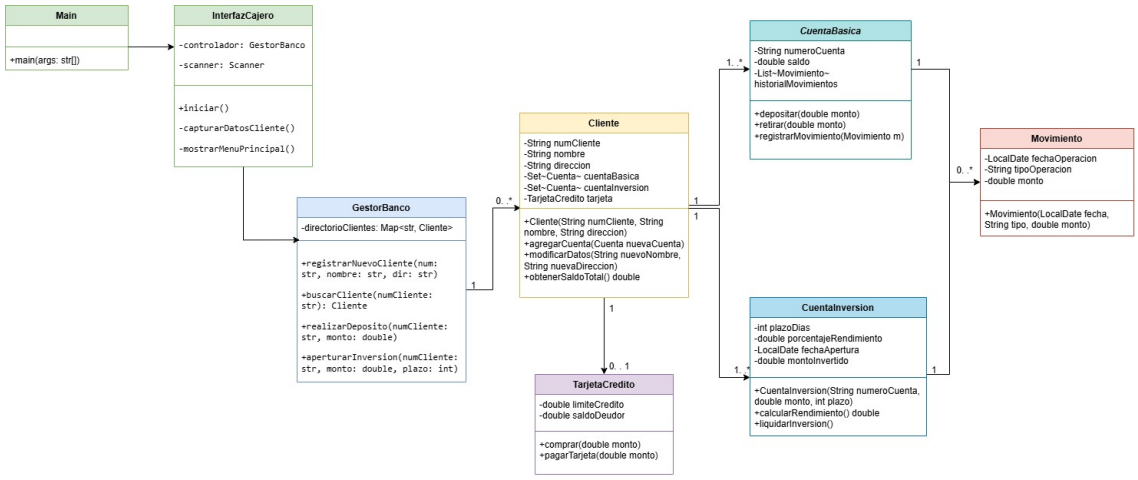

Figura 1: Diagrama UML inicial del Sistema Bancario

6.1. Cumplimiento y Desviaciones del Modelo Original

El sistema final cumple con la arquitectura de la Figura 1, con ligeros ajustes de modernización:

1. **Fusión de Interfaz:** La propuesta `InterfazCajero` fue absorbida por el método `main` de `SistemaBancario` para centralizar el `Scanner` y mantener bajo acoplamiento.

2. **Fechas Precisas:** En lugar de `LocalDate` para la clase `Movimiento`, el uso nativo de *timestamps* garantiza precisión milimétrica en auditorías, dándole un enfoque realista.

7. Conclusiones

La evolución desde la Versión 1 hasta esta entrega final demuestra cómo la correcta aplicación teórica del *Collections Framework* resuelve problemas de rendimiento en el mundo real. Se logró construir un núcleo transaccional robusto que no solo registra datos, sino que simula de manera precisa la estricta lógica financiera (como el fondeo condicionado) de una institución bancaria verdadera.